

AeroFit

April 16, 2025

0.1 About AeroFit

AeroFit is a leading brand in the field of fitness equipment. AeroFit provides a product range including machines such as treadmills, exercise bikes, gym equipment, and fitness accessories to cater to the needs of all categories of people.

0.2 Objective

The market research team at AeroFit wants to identify the characteristics of the target audience for each type of treadmill offered by the company, to provide a better recommendation of the treadmills to the new customers. The team decides to investigate whether there are differences across the product with respect to customer characteristics. Perform descriptive analytics to create a customer profile for each AeroFit treadmill product by developing appropriate tables and charts. For each AeroFit treadmill product, construct two-way contingency tables and compute all conditional and marginal probabilities along with their insights/impact on the business.

```
[16]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[ ]: !pip install numpy pandas matplotlib seaborn
```

```
[20]: df=pd.read_csv("https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/
↳001/125/original/aerofit_treadmill.csv")
```

```
[21]: df
```

```
[21]:
```

	Product	Age	Gender	Education	MaritalStatus	Usage	Fitness	Income	\
0	KP281	18	Male	14	Single	3	4	29562	
1	KP281	19	Male	15	Single	2	3	31836	
2	KP281	19	Female	14	Partnered	4	3	30699	
3	KP281	19	Male	12	Single	3	3	32973	
4	KP281	20	Male	13	Partnered	4	2	35247	
..	...	...	...	...	...	...	...	...	
175	KP781	40	Male	21	Single	6	5	83416	
176	KP781	42	Male	18	Single	5	4	89641	
177	KP781	45	Male	16	Single	5	5	90886	
178	KP781	47	Male	18	Partnered	4	5	104581	

```
179    KP781    48    Male        18    Partnered    4        5    95508
```

```
      Miles
0      112
1       75
2       66
3       85
4       47
..      ...
175    200
176    200
177    160
178    120
179    180
```

```
[180 rows x 9 columns]
```

```
[22]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 180 entries, 0 to 179
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Product         180 non-null   object
1   Age             180 non-null   int64
2   Gender          180 non-null   object
3   Education       180 non-null   int64
4   MaritalStatus   180 non-null   object
5   Usage           180 non-null   int64
6   Fitness         180 non-null   int64
7   Income          180 non-null   int64
8   Miles           180 non-null   int64
dtypes: int64(6), object(3)
memory usage: 12.8+ KB
```

```
[23]: df.shape
```

```
[23]: (180, 9)
```

```
[24]: df.ndim
```

```
[24]: 2
```

0.3 Features of the dataset:

Product Purchased: KP281, KP481, or KP781

Age: In years

Gender: Male/Female

Education: In years

MaritalStatus: Single or partnered

Usage: The average number of times the customer plans to use the treadmill each week.

Fitness: Self-rated fitness on a 1-to-5 scale, where 1 is the poor shape and 5 is the excellent shape.

Income: Annual income (in \$)

Miles: The average number of miles the customer expects to walk/run each week

0.4 Product Portfolio:

- The KP281 is an entry-level treadmill that sells for \$1,500.
- The KP481 is for mid-level runners that sell for \$1,750.
- The KP781 treadmill is having advanced features that sell for \$2,500.

```
[25]: df.describe().T
```

```
[25]:
```

	count	mean	std	min	25%	50%	\
Age	180.0	28.788889	6.943498	18.0	24.00	26.0	
Education	180.0	15.572222	1.617055	12.0	14.00	16.0	
Usage	180.0	3.455556	1.084797	2.0	3.00	3.0	
Fitness	180.0	3.311111	0.958869	1.0	3.00	3.0	
Income	180.0	53719.577778	16506.684226	29562.0	44058.75	50596.5	
Miles	180.0	103.194444	51.863605	21.0	66.00	94.0	

	75%	max
Age	33.00	50.0
Education	16.00	21.0
Usage	4.00	7.0
Fitness	4.00	5.0
Income	58668.00	104581.0
Miles	114.75	360.0

0.5 Insights:

Age - The age range of customers spans from 18 to 50 year, with an average age of 29 years.

Education - Customer education levels vary between 12 and 21 years, with an average education duration of 16 years.

Usage - Customers intend to utilize the product anywhere from 2 to 7 times per week, with an average usage frequency of 3 to 4 times per week.

Fitness - On average, customers have rated their fitness at 3 on a range of 5 points, reflecting a moderate level of fitness.

Income -The annual income of customers falls within the range of USD 30,000 to USD 100,000, with an average income of approximately USD 54,000.

Miles - Customers' weekly running goals range from 21 to 360 miles, with an average target of 103 miles per week count

```
[26]: df.describe(include=object).T
```

```
[26]:
```

	count	unique	top	freq
Product	180	3	KP281	80
Gender	180	2	Male	104
MaritalStatus	180	2	Partnered	107

0.6 Insights:

- Product- KP281 is the top product in sales in the duration of 3 months
- Gender- Most of the customers are male to buy products in 3 months
- Marital Status- Most of the customers are married to buy products in the last 3 months
- Checking all the columns and whether there are any null values

0.7 Checking all the columns and whether there are any null values

```
[27]: #Checking unique values for Product column
df['Product'].unique()
```

```
[27]: array(['KP281', 'KP481', 'KP781'], dtype=object)
```

```
[28]: #Checking null values for Product column
df['Product'].isnull().sum()
```

```
[28]: np.int64(0)
```

```
[29]: #Checking unique values for Age column
df['Age'].unique()
```

```
[29]: array([18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34,
        35, 36, 37, 38, 39, 40, 41, 43, 44, 46, 47, 50, 45, 48, 42])
```

```
[30]: #Checking null values for Age column
df['Age'].isnull().sum()
```

```
[30]: np.int64(0)
```

```
[31]: #Checking unique values for Gender column
df['Gender'].unique()
```

```
[31]: array(['Male', 'Female'], dtype=object)
```

```
[32]: #Checking null values for Gender column
df['Gender'].isnull().sum()
```

```
[32]: np.int64(0)
```

```
[33]: #Checking unique values for Education column
df['Education'].unique()
```

```
[33]: array([14, 15, 12, 13, 16, 18, 20, 21])
```

```
[34]: #Checking null values for Education column
df['Education'].isnull().sum()
```

```
[34]: np.int64(0)
```

```
[35]: #Checking unique values for Marital status column
df['MaritalStatus'].unique()
```

```
[35]: array(['Single', 'Partnered'], dtype=object)
```

```
[37]: #Checking null values for Product column
df['MaritalStatus'].isnull().sum()
```

```
[37]: np.int64(0)
```

```
[38]: #Checking unique values for Usage column
df['Usage'].unique()
```

```
[38]: array([3, 2, 4, 5, 6, 7])
```

```
[39]: #Checking null values for MaritalStatus column
df['Usage'].isnull().sum()
```

```
[39]: np.int64(0)
```

```
[40]: #Checking unique values for Fitness column
df['Fitness'].unique()
```

```
[40]: array([4, 3, 2, 1, 5])
```

```
[41]: #Checking null values for Fitness column
df['Fitness'].isnull().sum()
```

```
[41]: np.int64(0)
```

```
[42]: #Checking unique values for Income column
df['Income'].unique()
```

```
[42]: array([ 29562,  31836,  30699,  32973,  35247,  37521,  36384,  38658,
           40932,  34110,  39795,  42069,  44343,  45480,  46617,  48891,
           53439,  43206,  52302,  51165,  50028,  54576,  68220,  55713,
           60261,  67083,  56850,  59124,  61398,  57987,  64809,  47754,
           65220,  62535,  48658,  54781,  48556,  58516,  53536,  61006,
           57271,  52291,  49801,  62251,  64741,  70966,  75946,  74701,
           69721,  83416,  88396,  90886,  92131,  77191,  52290,  85906,
           103336,  99601,  89641,  95866, 104581,  95508])
```

```
[36]: #Checking null values for Income column
df['Income'].isnull().sum()
```

```
[36]: np.int64(0)
```

```
[37]: #Checking unique values for Miles column
df['Miles'].unique()
```

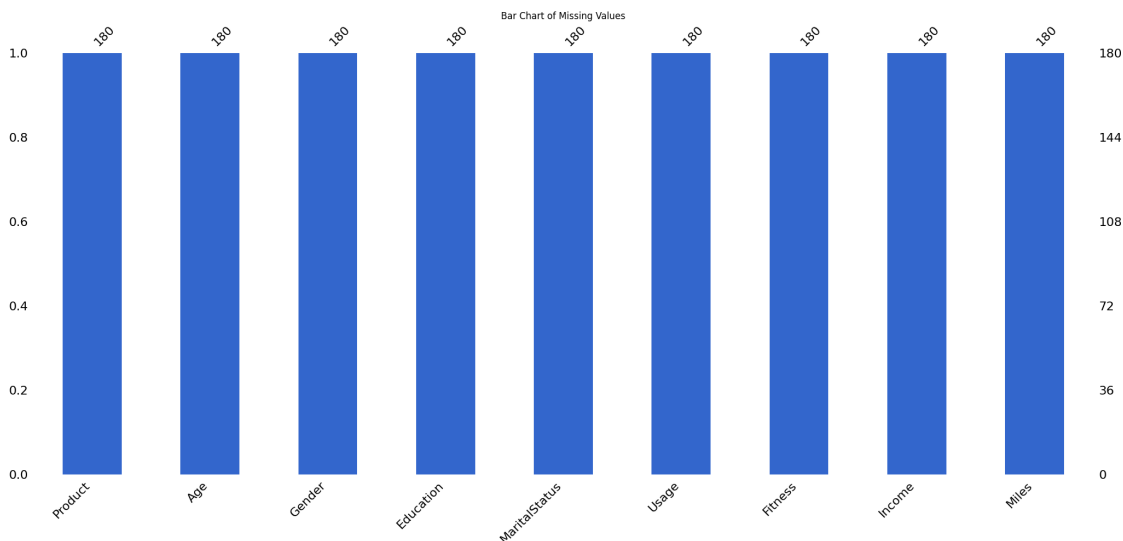
```
[37]: array([112,  75,  66,  85,  47, 141, 103,  94, 113,  38, 188,  56, 132,
           169,  64,  53, 106,  95, 212,  42, 127,  74, 170,  21, 120, 200,
           140, 100,  80, 160, 180, 240, 150, 300, 280, 260, 360])
```

```
[38]: #Checking null values for Miles column
df['Miles'].isnull().sum()
```

```
[38]: np.int64(0)
```

```
[ ]: !pip install missingno
```

```
[41]: import missingno as msno
msno.bar(df,color=(0.2,0.4,0.8))
plt.title('Bar Chart of Missing Values')
plt.show()
```



```
[53]: msno.dendrogram(df)
plt.title('Dendrogram of Missing Values')
plt.show()
```

C:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\scipy\cluster\hierarchy.py:2823: UserWarning: Attempting to set identical low and high ylims makes transformation singular; automatically expanding.

```
ax.set_ylim([dvw, 0])
```



- The line from the above plot shows that there are no null values in the dataset

0.8 Duplicate Detection

```
[42]: df.duplicated().sum()
```

```
[42]: np.int64(0)
```

- No duplicates found in the dataset

0.9 Univariate - categorical

```
[43]: #Categorical columns in the dataset are Product, Gender, MaritalStatus
df['Gender'].value_counts()
```

```
[43]: Gender
Male      104
Female    76
```

Name: count, dtype: int64

```
[58]: df['Product'].value_counts()
```

```
[58]: Product
      KP281      80
      KP481      60
      KP781      40
      Name: count, dtype: int64
```

```
[44]: df['MaritalStatus'].value_counts()
```

```
[44]: MaritalStatus
      Partnered    107
      Single        73
      Name: count, dtype: int64
```

```
[45]: import matplotlib.pyplot as plt
      # Assuming df is your DataFrame
      colors = plt.get_cmap('tab20c').colors
      plt.figure(figsize=(15,5))

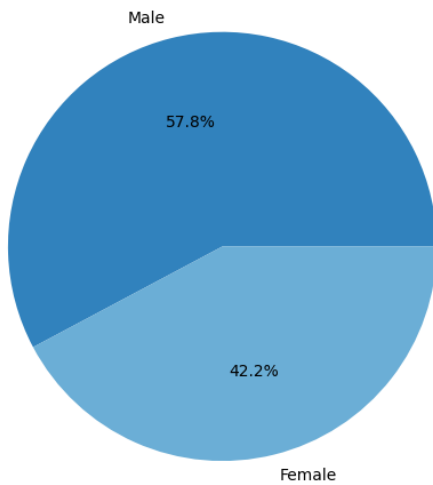
      # Gender Ratio Pie Chart
      plt.subplot(1, 3, 1)
      plt.pie(df['Gender'].value_counts(), labels=['Male', 'Female'], autopct='%1.1f%%', colors=colors)
      plt.title('Gender Ratio of Buying Aerofit Products')

      # Product Ratio Pie Chart
      plt.subplot(1, 3, 2)
      plt.pie(df['Product'].value_counts(), labels=['KP281', 'KP481', 'KP781'], autopct='%1.1f%%', colors=colors)
      plt.title('Product Ratio of Buying Aerofit Products')
      plt.tight_layout()
      plt.show()

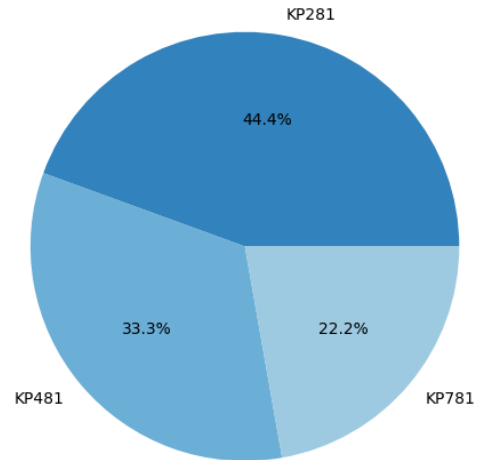
      # Marital Status Ratio Pie Chart
      plt.subplot(1, 3, 3)
      plt.pie(df['MaritalStatus'].value_counts(), labels=['Single', 'Partnered'], autopct='%1.1f%%', colors=colors)
      plt.title('Marital Status Ratio of Buying Aerofit Products')

      plt.tight_layout()
      plt.show()
```

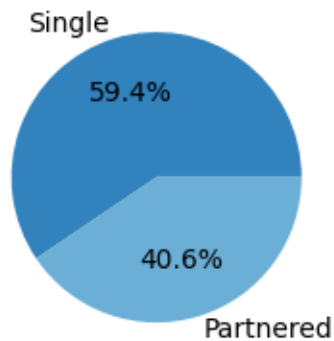

Gender Ratio of Buying Aerofit Products



Product Ratio of Buying Aerofit Products



Marital Status Ratio of Buying Aerofit Products



0.10 Insights:

Majority Male Buyers:

- 57.8% of the purchasers are male, indicating that a majority of Aerofit product buyers are men. Significant Female Buyers:
- 42.2% of the purchasers are female, showing that a significant portion of the customer base is female, although less than the male percentage.

Most Popular Product (KP281):

- 44.4% of the purchases are of the KP281 product, making it the most popular among the three products as it is the cheapest product available in the company.

Moderately Popular Product (KP481):

- 33.3% of the purchases are of the KP481 product, indicating that it is also fairly popular but less so than KP281.

Least Popular Product (KP781):

- 22.2% of the purchases are of the KP781 product, making it the least popular among the three as it is the most expensive product available in the company

Majority Single Buyers:

- 59.4% of the purchasers are single, indicating that a majority of Aero t product buyers are not in a partnered relationship.

Significant Partnered Buyers:

- 40.6% of the purchasers are partnered, showing that a substantial portion of the customer base is in a relationship

```
[66]: # Barplot for categorical columns
import matplotlib.pyplot as plt
# Assuming df is your DataFrame
plt.figure(figsize=(15, 5))
# Product Count Bar Chart
plt.subplot(1, 3, 1)
bars = plt.bar(df['Product'].value_counts().index, df['Product'].
    ↪value_counts(), color='#1E90FF')
plt.title('Product Count')
plt.xlabel('Product')
plt.ylabel('Count')

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, yval, int(yval), ha='center',
    ↪va='bottom')

# Gender Count Bar Chart
plt.subplot(1, 3, 2)
bars = plt.bar(df['Gender'].value_counts().index, df['Gender'].value_counts(),
    ↪color='#008000')
plt.title('Gender Count')
plt.xlabel('Gender')
plt.ylabel('Count')

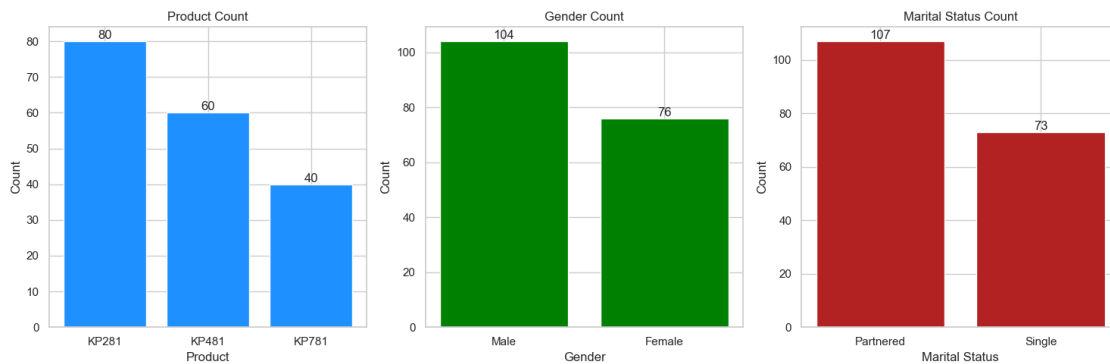
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, yval, int(yval), ha='center',
    ↪va='bottom')
```

```

# Marital Status Count Bar Chart
plt.subplot(1, 3, 3)
bars = plt.bar(df['MaritalStatus'].value_counts().index, df['MaritalStatus'].
    ↪value_counts(), color='#B22222')
plt.title('Marital Status Count')
plt.xlabel('Marital Status')
plt.ylabel('Count')

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, yval, int(yval), ha='center', ↪
    ↪va='bottom')
plt.tight_layout()
plt.show()

```



Product Popularity:

- KP781 is the most popular product with the highest count of 80, followed by KP281 with 60, and KP481 with 48.

Gender Distribution:

- There are more males (104) than females (76) represented in the data.

Marital Status:

- Single individuals (73) outnumber partnered ones (37), suggesting a higher purchase rate among single

0.11 Univariate - Numerical

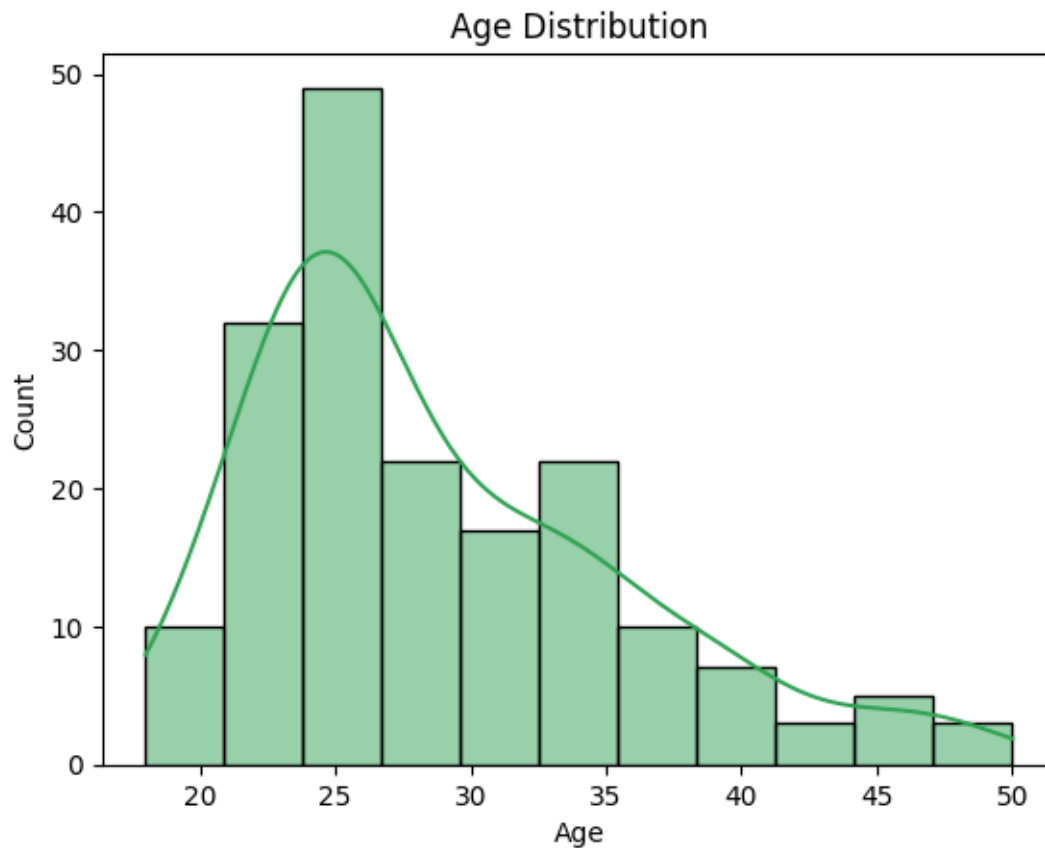
```
[48]: df.describe().T
```

```
[48]:
```

	count	mean	std	min	25%	50%	\
Age	180.0	28.788889	6.943498	18.0	24.00	26.0	
Education	180.0	15.572222	1.617055	12.0	14.00	16.0	
Usage	180.0	3.455556	1.084797	2.0	3.00	3.0	
Fitness	180.0	3.311111	0.958869	1.0	3.00	3.0	
Income	180.0	53719.577778	16506.684226	29562.0	44058.75	50596.5	
Miles	180.0	103.194444	51.863605	21.0	66.00	94.0	

	75%	max
Age	33.00	50.0
Education	16.00	21.0
Usage	4.00	7.0
Fitness	4.00	5.0
Income	58668.00	104581.0
Miles	114.75	360.0

```
[49]: sns.histplot(x='Age',data=df,kde=True,color=plt.get_cmap('tab20c').colors[8])
plt.title('Age Distribution')
plt.show()
```



0.12 Insights:

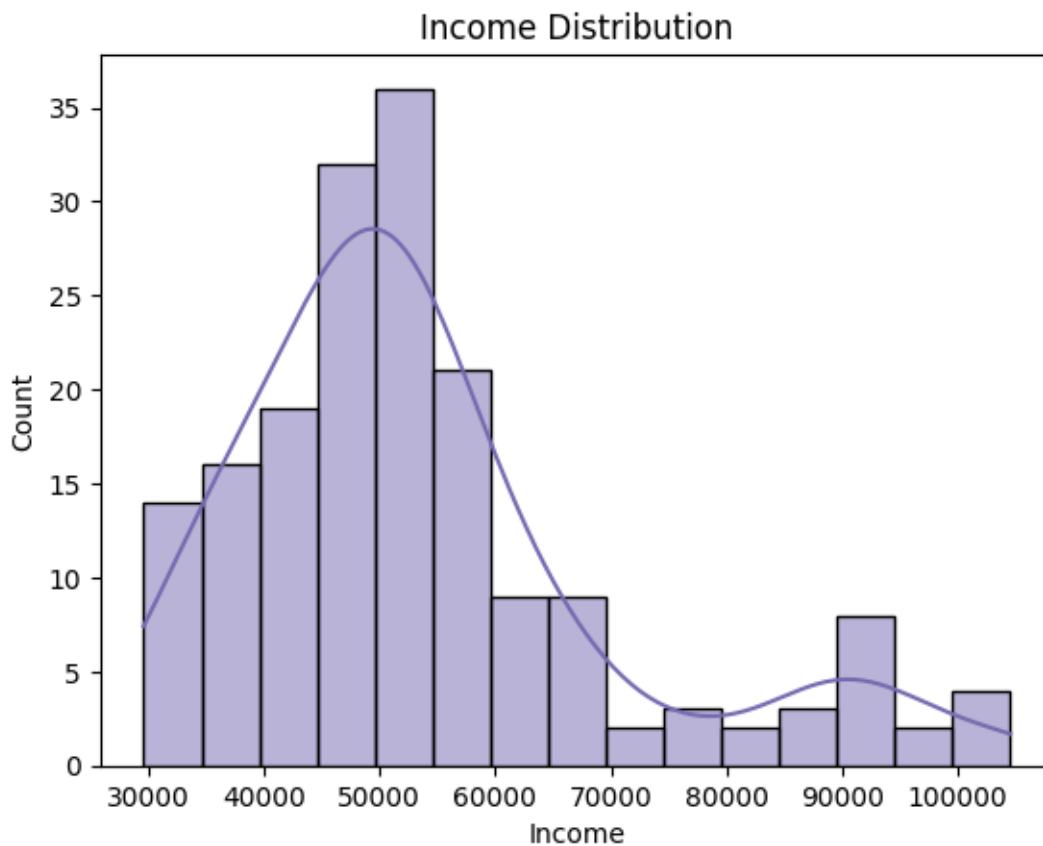
Peak Age Group:

- The most significant number of individuals falls within the 25-30 years age range, as indicated by the peak count exceeding 50.

Trend Analysis:

- The line graph shows a smoothed trend, peaking around the same age group and then gradually declining, suggesting a decrease in the represented demographic as age increases

```
[50]: sns.histplot(x='Income',data=df,kde=True,color=plt.get_cmap('tab20c').  
→ colors[12])  
plt.title('Income Distribution')  
plt.show()
```



0.13 Insights:

- Most Common Income Range- The majority of individuals earn between 50,000 and 60,000. This is indicated by the tallest bar and the peak of the line graph

```
[51]: import matplotlib.pyplot as plt
import seaborn as sns

# Assuming df is your DataFrame
plt.figure(figsize=(15, 10))
sns.set(style="whitegrid")

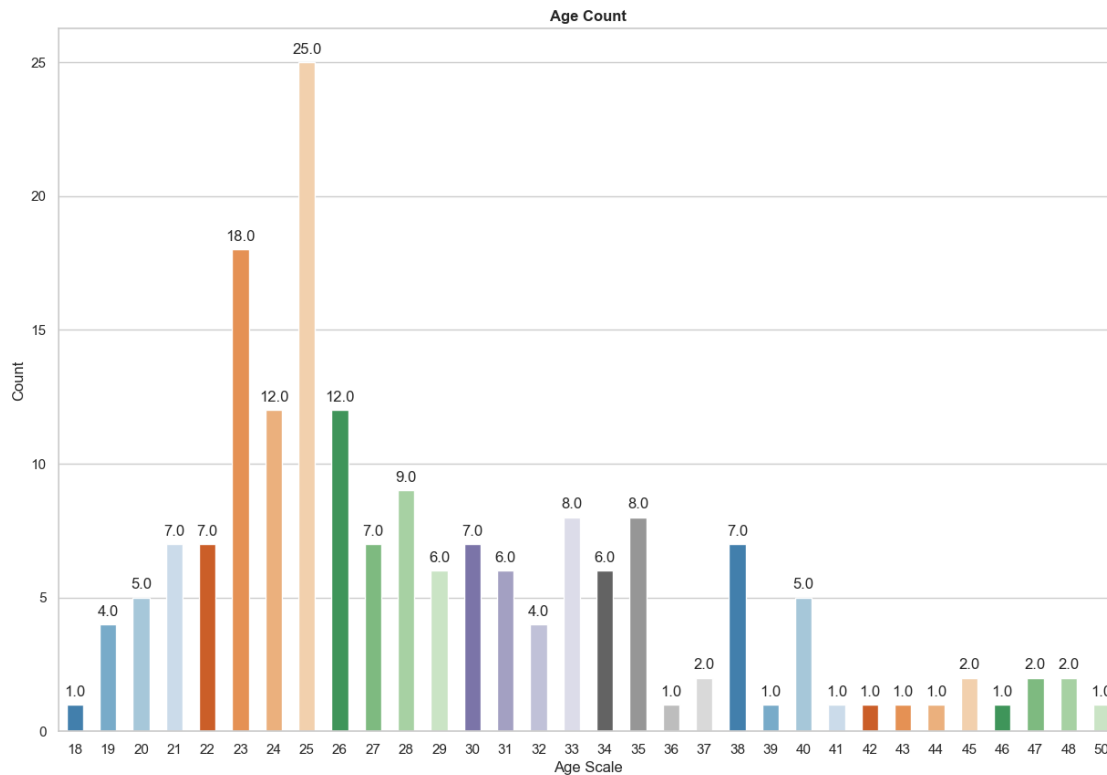
# Fix here: generate the correct number of colors for unique 'Age' values
color = sns.color_palette("tab20c", n_colors=df['Age'].nunique())

# Apply fix: hue='Age' + legend=False, and valid palette
ax = sns.countplot(x='Age', data=df, hue='Age', palette=color, width=0.5,
    ↪legend=False)

# Add title and labels
plt.title('Age Count', weight='bold')
plt.xlabel('Age Scale')
plt.ylabel('Count')

# Add count labels on top of the bars
for p in ax.patches:
    ax.annotate(f'{p.get_height()}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 10), textcoords='offset points')

plt.show()
```



0.14 Insights:

- Age scale of 25 there are many people who works out using threadmill

```
[53]: import matplotlib.pyplot as plt
import seaborn as sns

# Assuming df is your DataFrame
plt.figure(figsize=(7, 5))
sns.set(style="whitegrid")

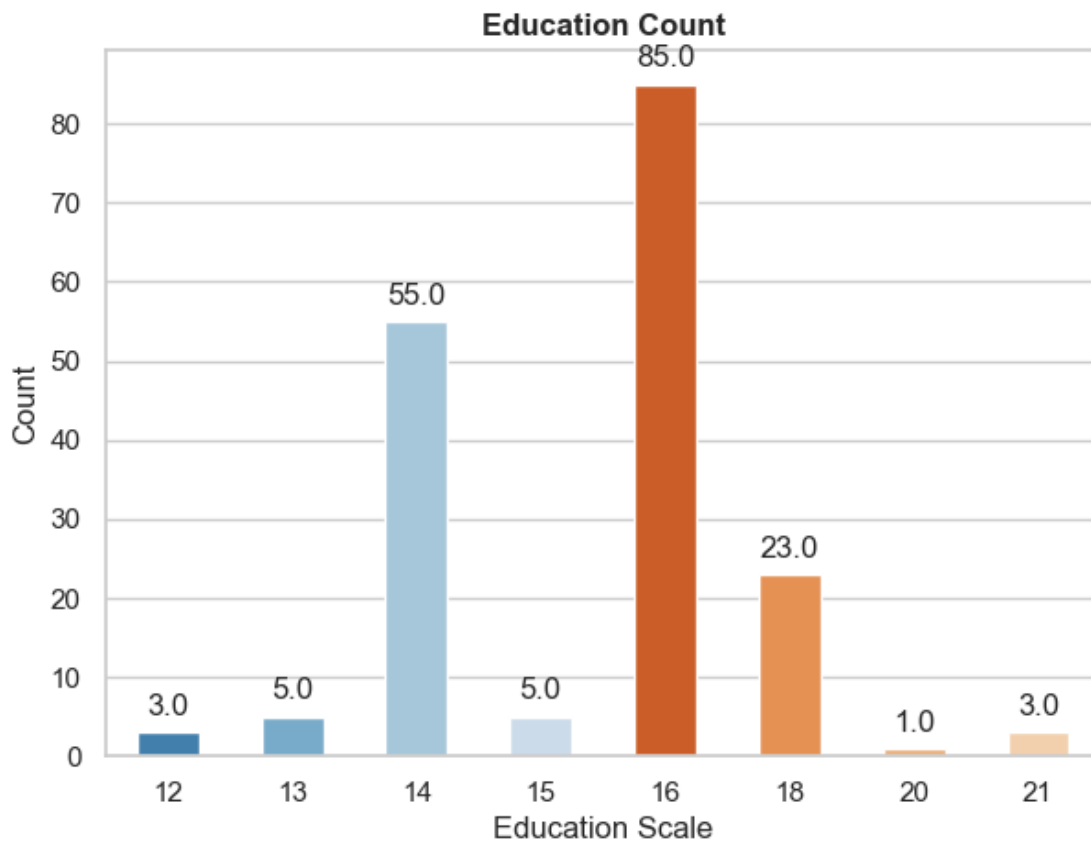
# FIXED: use seaborn's color_palette to match number of unique categories
color = sns.color_palette("tab20c", n_colors=df['Education'].nunique())

# FIXED: add hue and legend=False to match future seaborn behavior
ax = sns.countplot(x='Education', data=df, hue='Education', palette=color, width=0.5, legend=False)

# Add title and labels
plt.title('Education Count', weight='bold')
plt.xlabel('Education Scale')
plt.ylabel('Count')
```

```
# Add count labels on top of the bars
for p in ax.patches:
    ax.annotate(f'{p.get_height()}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 10), textcoords='offset points')

plt.show()
```



0.15 Insights:

- Most educated people who workout has education over 16 years.
- Least educated people who workout has education over 12 years

```
[54]: import matplotlib.pyplot as plt
import seaborn as sns
# Assuming df is your DataFrame
plt.figure(figsize=(7, 5))
```



```

sns.set(style="whitegrid")

# FIX: Use a valid palette with correct number of unique 'Usage' values
color = sns.color_palette("tab20c", n_colors=df['Usage'].nunique())

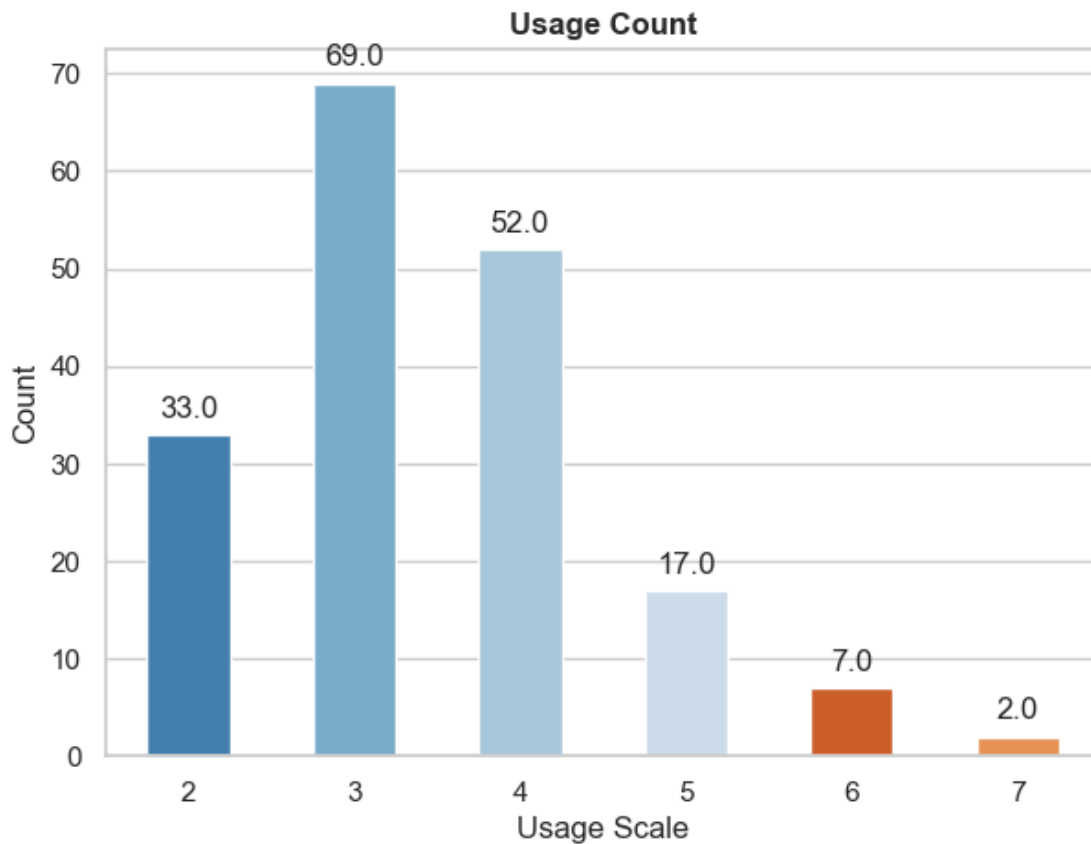
# FIX: Add hue and suppress legend to fix the FutureWarning
ax = sns.countplot(x='Usage', data=df, hue='Usage', palette=color, width=0.5,
    legend=False)

# Add title and labels
plt.title('Usage Count', weight='bold')
plt.xlabel('Usage Scale')
plt.ylabel('Count')

# Add count labels on top of the bars
for p in ax.patches:
    ax.annotate(f'{p.get_height()}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 10), textcoords='offset points')

plt.show()

```



0.16 Insights:

- Most of the people workouts 3 times a week frequently and there are 69 people who workouts on treadmill.
- 2 people who are the most focussed to workout on treadmill

```
[55]: import matplotlib.pyplot as plt
import seaborn as sns

# Assuming df is your DataFrame
plt.figure(figsize=(6, 5))
sns.set(style="whitegrid")

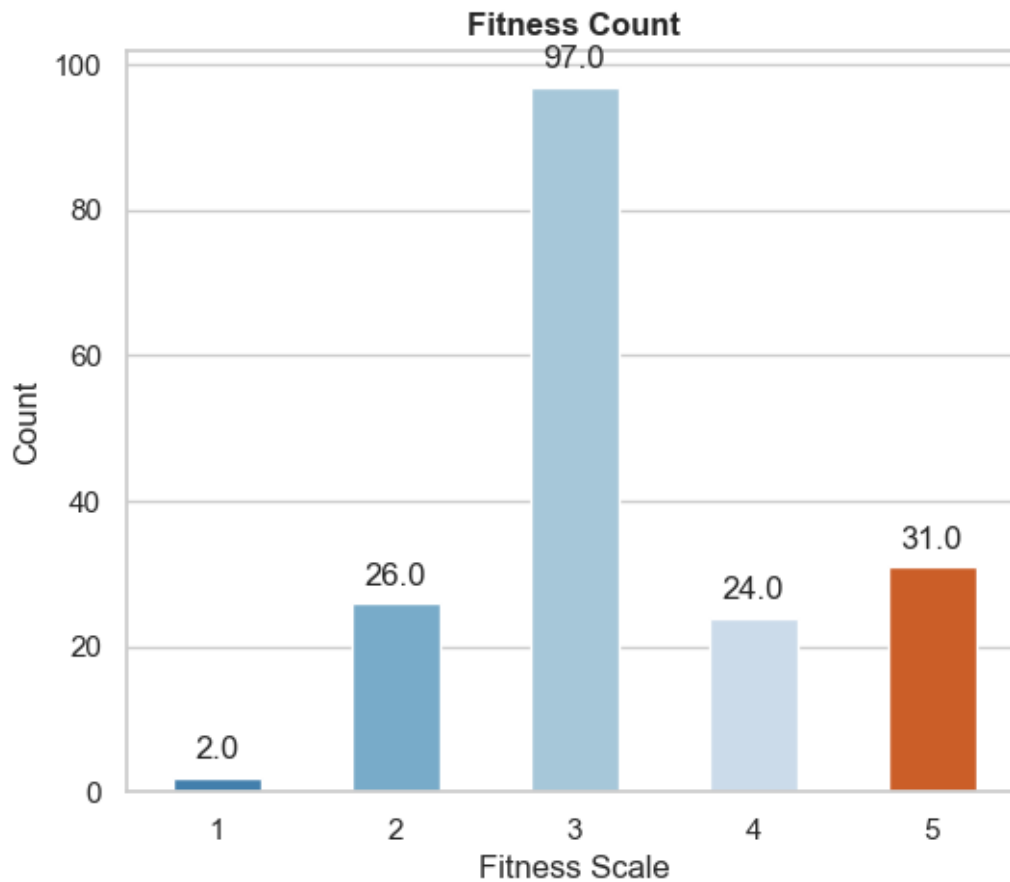
# FIX: Generate palette matching number of unique Fitness values
color = sns.color_palette("tab20c", n_colors=df['Fitness'].nunique())

# FIX: Add hue + legend=False
ax = sns.countplot(x='Fitness', data=df, hue='Fitness', palette=color, width=0.
↳5, legend=False)

# Add title and labels
plt.title('Fitness Count', weight='bold')
plt.xlabel('Fitness Scale')
plt.ylabel('Count')

# Add count labels on top of the bars
for p in ax.patches:
    ax.annotate(f'{p.get_height()}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 10), textcoords='offset points')

plt.show()
```



0.17 Insights:

- Most of the people are rated 3 out of 5 star

```
[143]: import matplotlib.pyplot as plt
import seaborn as sns

# Assuming df is your DataFrame
plt.figure(figsize=(15, 10))
sns.set(style="whitegrid")

# FIXED: generate palette based on unique 'Miles' values
color = sns.color_palette("tab20c", n_colors=df['Miles'].nunique())

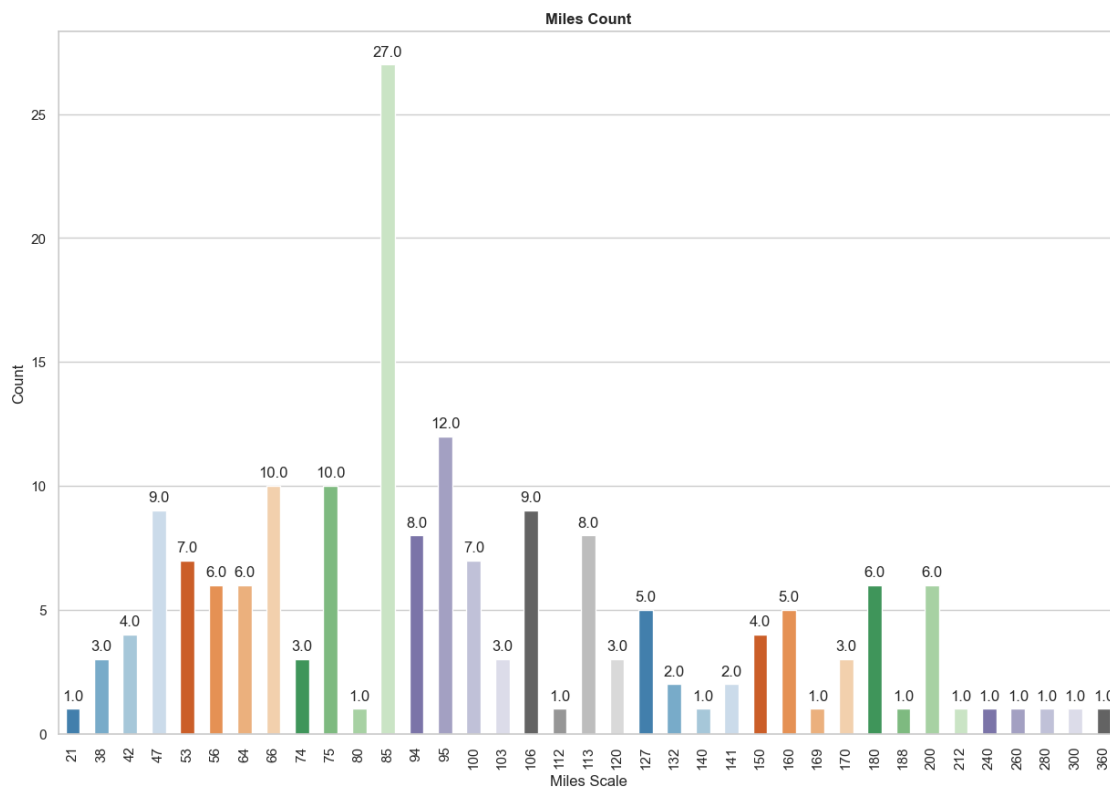
# FIXED: added hue + legend=False to silence FutureWarning
ax = sns.countplot(x='Miles', data=df, hue='Miles', palette=color, width=0.5,
↳ legend=False)

# Add title and labels
```

```
plt.title('Miles Count', weight='bold')
plt.xlabel('Miles Scale')
plt.ylabel('Count')
plt.xticks(rotation=90)

# Add count labels on top of the bars
for p in ax.patches:
    ax.annotate(f'{p.get_height()}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 10), textcoords='offset points')

plt.show()
```



0.18 Insights:

- 27 people ran around 85 miles in a week

```
[56]: import matplotlib.pyplot as plt

# List of numerical columns
num_cols = ['Age', 'Education', 'Usage', 'Fitness', 'Income', 'Miles']
```

```

colors = ['skyblue', 'lightgreen', 'lightcoral', 'plum', 'khaki', 'lightpink']

plt.figure(figsize=(15, 10))

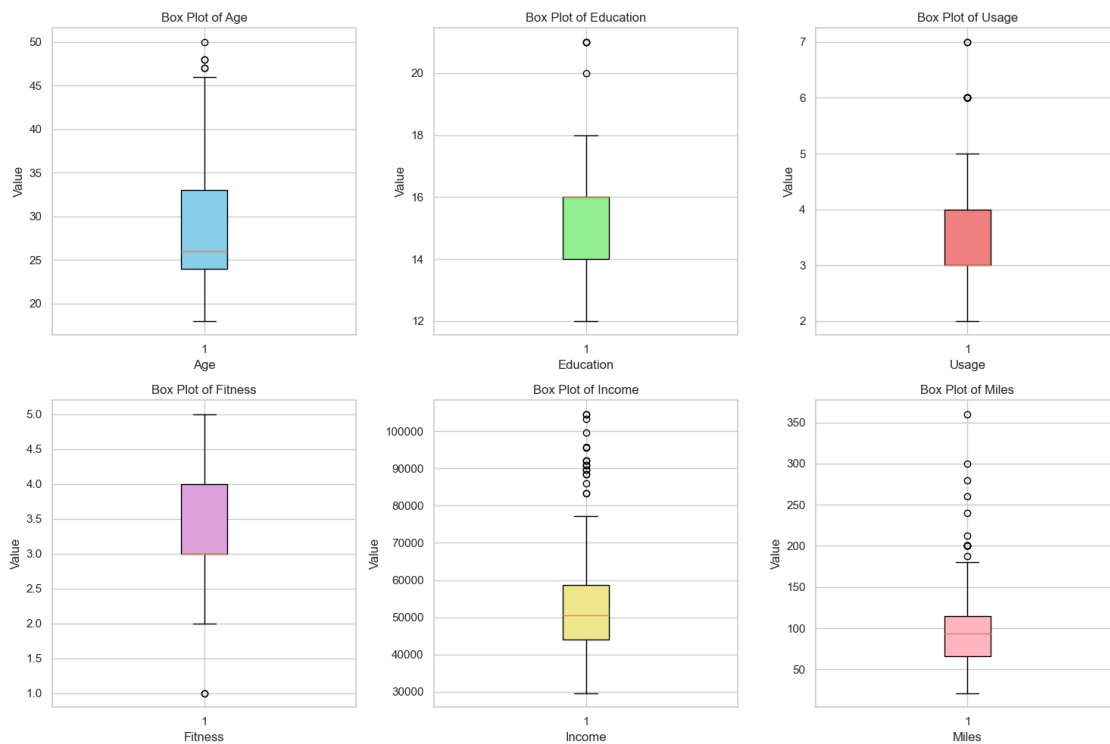
# Loop through each column and plot
for i, col in enumerate(num_cols, 1):
    plt.subplot(2, 3, i)
    box = plt.boxplot(df[col], patch_artist=True) # Allow coloring

    # Set color
    for patch in box['boxes']:
        patch.set_facecolor(colors[i-1])

    plt.xlabel(col)
    plt.ylabel('Value')
    plt.title(f'Box Plot of {col}')

plt.tight_layout()
plt.show()

```



0.19 Insights

Age: The median age is around 25. There are several outliers above 45.

Education: The median education level is around 14. There are outliers above 18.

Usage: The median usage is around 3. There are a few outliers above 6.

Fitness: The median fitness level is around 3.5. There are outliers below 1.5.

Income: The median income is around 50,000. There are several outliers above 90,000.

Miles: The median miles is around 100. There are several outliers above 200.

Clipping the data with 5 percentile and 95 percentile.

```
[57]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Example DataFrame creation (if needed)
# df = pd.DataFrame({...})

continuous_vars = ['Age', 'Education', 'Usage', 'Fitness', 'Income', 'Miles']
colors = ['skyblue', 'lightgreen', 'lightcoral', 'plum', 'khaki', 'lightpink']
    ↪ # Colors for each plot

# Detecting outliers and clipping data between 5th and 95th percentiles
for var in continuous_vars:
    # Calculate the 5th and 95th percentiles
    lower_bound = np.percentile(df[var], 5)
    upper_bound = np.percentile(df[var], 95)
    # Clip the data
    df[var] = np.clip(df[var], lower_bound, upper_bound)
    # Print the bounds for verification
    print(f'{var}: 5th Percentile = {lower_bound:.2f}, 95th Percentile = {upper_bound:.2f}')
    ↪ # Color

# Plot boxplots again to show the effect of clipping
plt.figure(figsize=(15, 10))
for i, var in enumerate(continuous_vars):
    plt.subplot(2, 3, i + 1)
    box = plt.boxplot(df[var], patch_artist=True) # <-- Enables coloring
    for patch in box['boxes']:
        patch.set_facecolor(colors[i]) # <-- Sets individual box color
    plt.title(f'Box Plot of {var} (Clipped)')
    plt.xlabel(var)
    plt.ylabel('Value')
plt.tight_layout()
plt.show()
```

Age: 5th Percentile = 20.00, 95th Percentile = 43.05

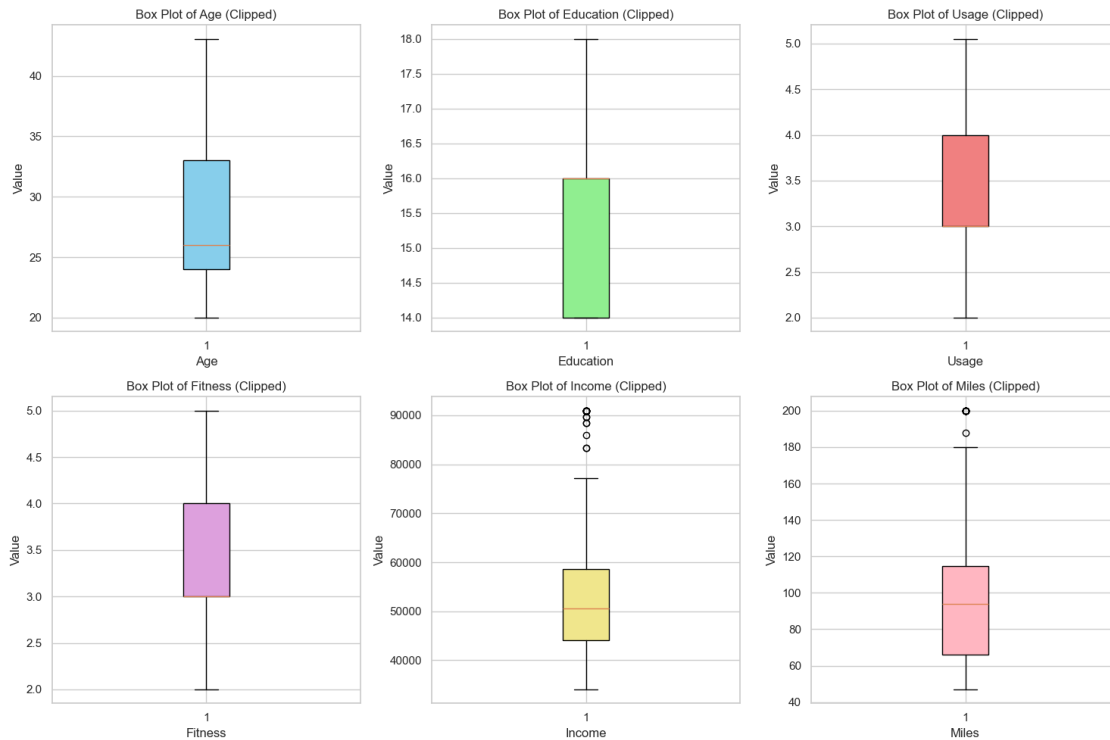
Education: 5th Percentile = 14.00, 95th Percentile = 18.00

Usage: 5th Percentile = 2.00, 95th Percentile = 5.05

Fitness: 5th Percentile = 2.00, 95th Percentile = 5.00

Income: 5th Percentile = 34053.15, 95th Percentile = 90948.25

Miles: 5th Percentile = 47.00, 95th Percentile = 200.00



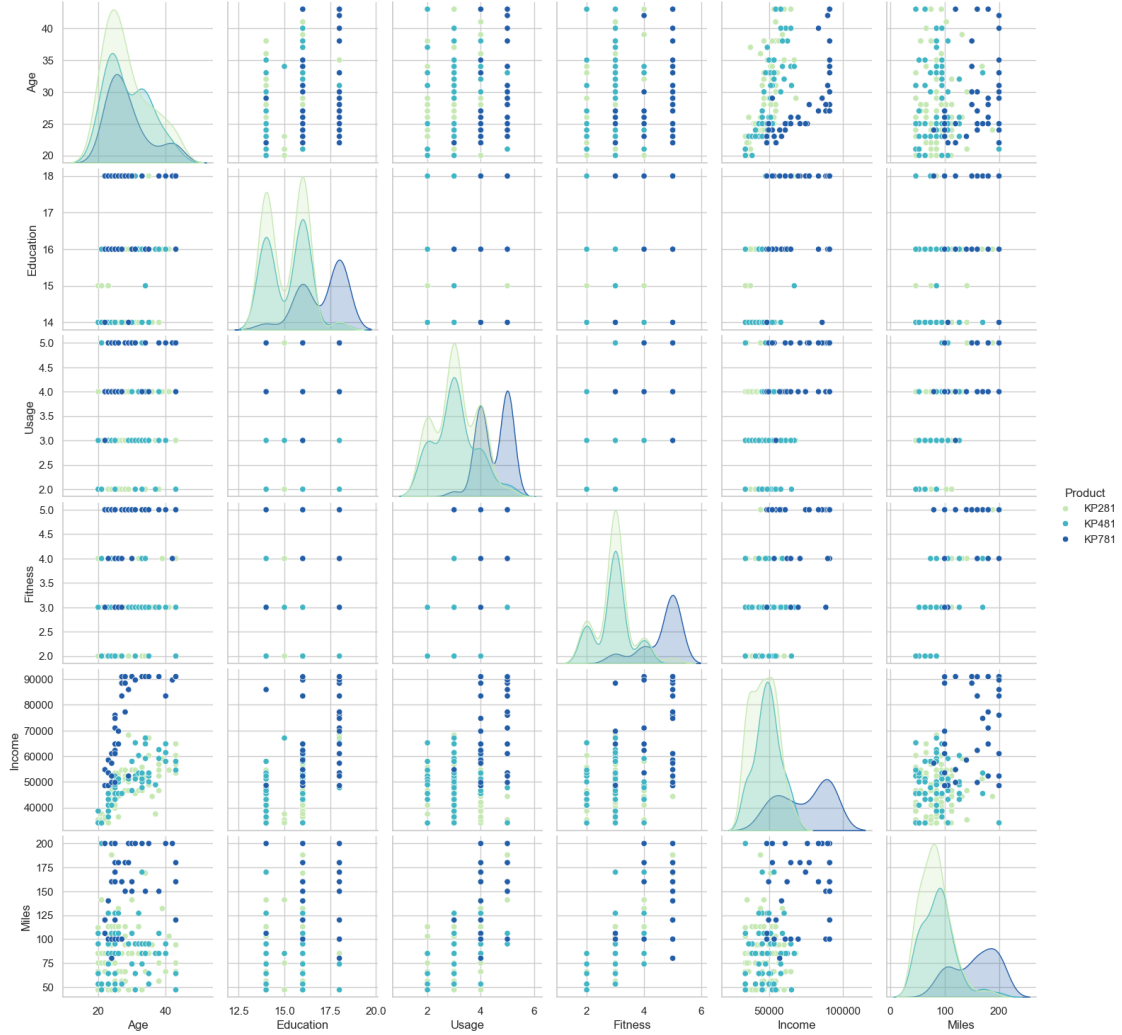
0.20 General Observations:

Outliers: The initial plots show numerous outliers for each variable, particularly in the upper range for Age, Education, Usage, Income, and Miles, and in the lower range for Fitness.

Distribution: After clipping, the data distributions are more compact, and the extreme values (outliers) are no longer present.

Correlation Between Continuous Variables

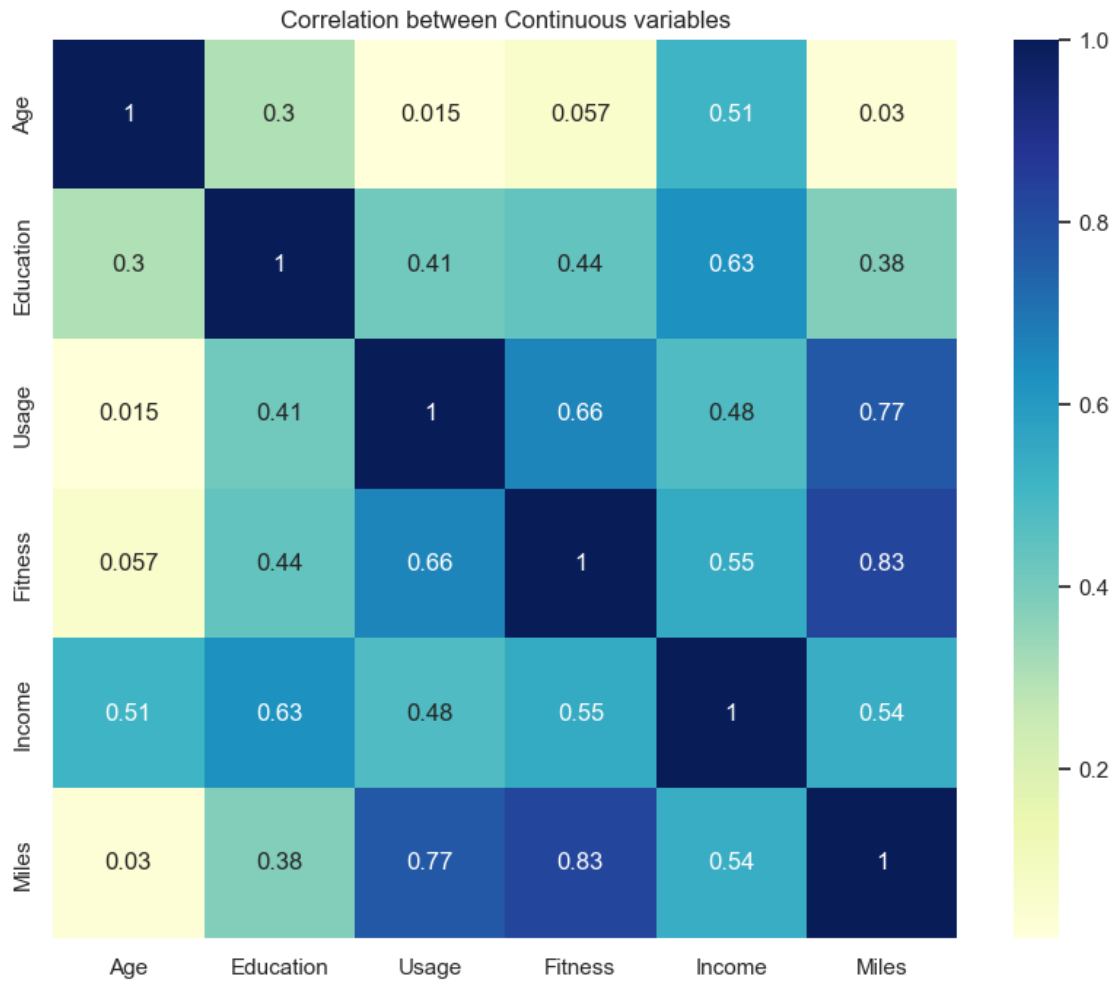
```
[60]: #Correlation between continuous variables pairplot
#converting age, usage, fitness to int64
df['Age']=df['Age'].astype('int64')
df['Usage']=df['Usage'].astype('int64')
df['Fitness']=df['Fitness'].astype('int64')
sns.pairplot(df, hue='Product', palette='YlGnBu')
plt.show()
```



0.21 Insights:

- KP281: Attracts younger users with lower education, tness, income, and miles traveled
- KP481: Appeals to a more diverse age group with higher tness and income levels, and moderate travel distances.
- KP781: Preferred by older users with higher income and tness levels, and longer travel distances
- From above plots we can clearly distinguish the user of KP781 based on Fitness, Miles, Income and Usage
- The users of KP281 and KP481 are similar in pattern. This will require a deeper analysis to differentiate between the two


```
[61]: plt.figure(figsize=(10,8))
corr_mat=df[continuous_vars].corr()
sns.heatmap(corr_mat,annot=True,cmap='YlGnBu')
plt.title('Correlation between Continuous variables')
plt.show()
```



0.22 Insights:

Strong Positive Correlations:

- Fitness and Miles: High correlation (0.83), indicating that higher fitness levels are associated with more miles
- Usage and Miles: One of the high correlation (0.77), suggesting that increased usage is linked to more miles

Moderate Positive Correlations:

- Education and Income: Correlation of (0.63), showing that higher education levels are associated with higher income.
- Usage and Fitness: Correlation of (0.66), indicating that higher usage is related to better fitness.
- Age and Income: Correlation of (0.51), suggesting a weak positive relationship between age and income

Low to Moderate Correlations:

- Age and Education: Correlation of (0.3), indicating a weak positive relationship between age and education. Weak or No Correlations:
- Most other variable pairs show weak or no significant correlation, indicating little to no linear relationship between them

0.23 Probability

Find the marginal probability (what percent of customers have purchased KP281, KP481, or KP781)

```
[63]: length_of_df=len(df)
      prob_KP281=round(len(df[df['Product']=='KP281'])/length_of_df,2)
      prob_KP481=round(len(df[df['Product']=='KP481'])/length_of_df,2)
      prob_KP781=round(len(df[df['Product']=='KP781'])/length_of_df,2)
      print('Probability of KP281:',prob_KP281)
      print('Probability of KP481:',prob_KP481)
      print('Probability of KP781:',prob_KP781)
```

Probability of KP281: 0.44

Probability of KP481: 0.33

Probability of KP781: 0.22

Marginal Probability of Each Product:

→ 44.44% of customers bought KP281

→ 33.33% of customers bought KP481

→ 22.22% of customers bought KP7

```
[64]: #binned for age [18to25,25 to35,35to45]
      age_bins=[18,25,35,45]
      age_labels=['Young Adults','Adults','Middle Aged']
      df['Age_group']=pd.cut(df['Age'],bins=age_bins,labels=age_labels)

      #Binning for education [0to12,12to14,14to16,>16]
      education_bins=[12,14,16,float('inf')]
      education_labels=['Primary','Secondary','Higher']
      df['Edu_group']=pd.
      ↪cut(df['Education'],bins=education_bins,labels=education_labels)
```

```

#Binning for income [0to40k,40kto60k,60kto80k,>80k]
income_bins=[0,40000,60000,80000,float('inf')]
income_labels=['Low Income','Moderate Income','High Income','Very High Income']
df['Income_group']=pd.cut(df['Income'],bins=income_bins,labels=income_labels)

#binning for usage[0-2,2-3,>4]
usage_bin=[0,2,3,4,float('inf')]
usage_labels=['Infrequent','Regular','Frequent','Power']
df['Usage_group']=pd.cut(df['Usage'],bins=usage_bin,labels=usage_labels)

#Binning for Fitness [0-2,3-4,>4]
fitness_bins=[0,2,3,4,float('inf')]
fitness_labels=['Poor','Below Average','Average','Excellent']
df['Fitness_group']=pd.
    ↪cut(df['Fitness'],bins=fitness_bins,labels=fitness_labels)

#Binning for miles [0-50,50-100,100-150,>150]
miles_bin = [0, 50, 100, 150, float('inf')]
miles_label = ['Low ','Moderate', 'High', 'Very High']
df['Miles_group']=pd.cut(df['Miles'],bins=miles_bin,labels=miles_label)
df

```

```

[64]:
   Product  Age  Gender  Education  MaritalStatus  Usage  Fitness  Income  \
0    KP281   20   Male         14         Single      3        4  34053.15
1    KP281   20   Male         15         Single      2        3  34053.15
2    KP281   20  Female         14    Partnered      4        3  34053.15
3    KP281   20   Male         14         Single      3        3  34053.15
4    KP281   20   Male         14    Partnered      4        2  35247.00
..     ...   ...   ...         ...         ...     ...     ...
175   KP781   40   Male         18         Single      5        5  83416.00
176   KP781   42   Male         18         Single      5        4  89641.00
177   KP781   43   Male         16         Single      5        5  90886.00
178   KP781   43   Male         18    Partnered      4        5  90948.25
179   KP781   43   Male         18    Partnered      4        5  90948.25

   Miles  Age_group  Edu_group  Income_group  Usage_group  \
0     112  Young Adults  Primary    Low Income    Regular
1      75  Young Adults  Secondary  Low Income  Infrequent
2      66  Young Adults  Primary    Low Income    Frequent
3      85  Young Adults  Primary    Low Income    Regular
4      47  Young Adults  Primary    Low Income    Frequent
..     ...   ...         ...         ...     ...
175    200  Middle Aged  Higher  Very High Income    Power
176    200  Middle Aged  Higher  Very High Income    Power
177    160  Middle Aged  Secondary  Very High Income    Power
178    120  Middle Aged  Higher  Very High Income    Frequent

```

179 180 Middle Aged Higher Very High Income Frequent

	Fitness_group	Miles_group
0	Average	High
1	Below Average	Moderate
2	Below Average	Moderate
3	Below Average	Moderate
4	Poor	Low
..	...	...
175	Excellent	Very High
176	Average	Very High
177	Excellent	Very High
178	Excellent	High
179	Excellent	Very High

[180 rows x 15 columns]

```
[177]: #Conditional Probability of Product with respect to Age
age_bins=[18,25,35,45]
df['Age_group']=pd.cut(df['Age'],bins=age_bins,labels=age_labels)
cond_prob_age=np.round(pd.
    ↪crosstab(df['Age_group'],df['Product'],normalize=True,margins=True),2)
cond_prob_age
```

```
[177]: Product      KP281  KP481  KP781  All
Age_group
Young Adults    0.19   0.16   0.09  0.44
Adults          0.18   0.13   0.09  0.41
Middle Aged     0.08   0.04   0.03  0.16
All             0.44   0.33   0.22  1.00
```

Marginal Probability of each Age_Group buying Threadmill:

- The Probability of a treadmill being purchased by a Young Adult(18-25) is 44%
- The Probability of a treadmill being purchased by a Adults(26-35) is 41%
- The Probability of a treadmill being purchased by a Middle Aged(36-45) is 16%

Conditional Probability:

- The conditional probability of purchasing the treadmill model given that the customer is Young Adult is For Treadmill model KP281 - 19% For Treadmill model KP481 - 16% For Treadmill model KP781 - 9%
- The conditional probability of purchasing the treadmill model given that the customer is Adult is - For Treadmill model KP281 - 18% For Treadmill model KP481 - 13% For Treadmill model KP781 - 9%

- The conditional probability of purchasing the treadmill model given that the customer is Middle Aged is - For Treadmill model KP281 - 8% For Treadmill model KP481 - 4% For Treadmill model KP781 - 3%

```
[65]: #Gender
cond_prob_gender=np.round(pd.
↪crosstab(df['Gender'],df['Product'],normalize=True,margins=True),2)
cond_prob_gender
```

```
[65]: Product  KP281  KP481  KP781  All
Gender
Female    0.22   0.16   0.04  0.42
Male      0.22   0.17   0.18  0.58
All        0.44   0.33   0.22  1.00
```

Marginal Probability of each Gender buying Threadmill:

- The Probability of threadmill being purchased by Female is 42%
- The Probability of threadmill being purchased by Male is 58%

Conditional Probability:

- The conditional probability of purchasing the treadmill model given that the customer is Female is - For Treadmill model KP281 - 22% For Treadmill model KP481 - 16% For Treadmill model KP781 - 4%
- The conditional probability of purchasing the treadmill model given that the customer is Male is - For Treadmill model KP281 - 22% For Treadmill model KP481 - 17% For Treadmill model KP781 - 18%

```
[179]: #Edu_grp
cond_prob_edu=np.round(pd.
↪crosstab(df['Edu_group'],df['Product'],normalize=True,margins=True),2)
cond_prob_edu
```

```
[179]: Product    KP281  KP481  KP781  All
Edu_group
Primary    0.19   0.14   0.01  0.35
Secondary  0.24   0.18   0.08  0.50
Higher     0.01   0.01   0.13  0.15
All        0.44   0.33   0.22  1.00
```

Marginal Probability of each Educated Person buying Threadmill:

- The Probability of threadmill being purchased by Primary Educated Person(0-12) is 35%
- The Probability of threadmill being purchased by Secondary Educated Person(13-14) is 50%
- The Probability of threadmill being purchased by Higher Educated Person(above 14) is 15%

Conditional Probability:

- The conditional probability of purchasing the treadmill model given that the customer is Primary Educated Person is - For Treadmill model KP281 - 19% In []: Out[]: For Treadmill model KP481 - 14% For Treadmill model KP781 - 1%
- The conditional probability of purchasing the treadmill model given that the customer is Secondary Educated Person is - For Treadmill model KP281 - 24% For Treadmill model KP481 - 18% For Treadmill model KP781 - 8%
- The conditional probability of purchasing the treadmill model given that the customer is Higher Educated Person is - For Treadmill model KP281 - 1% For Treadmill model KP481 - 1% For Treadmill model KP781 - 13%

```
[181]: #Marital_status
cond_prob_marital=np.round(pd.
    ↪crosstab(df['MaritalStatus'],df['Product'],normalize=True,margins=True),2)
cond_prob_marital
```

```
[181]: Product      KP281  KP481  KP781   All
MaritalStatus
Partnered      0.27    0.20    0.13  0.59
Single         0.18    0.13    0.09  0.41
All            0.44    0.33    0.22  1.00
```

Marginal Probability of Partnered and Single buying Threadmill:

- The Probability of threadmill being purchased by Partnered is 59%
- The Probability of threadmill being purchased by Single is 41%

Conditional Probability:

- The conditional probability of purchasing the treadmill model given that the customer is Partnered Person is - For Treadmill model KP281 24% For Treadmill model KP481 - 18% For Treadmill model KP781 - 8% In []: Out[]:
- The conditional probability of purchasing the treadmill model given that the customer is Single Person is - For Treadmill model KP281 24% For Treadmill model KP481 - 18% For Treadmill model KP781 - 8%

```
[182]: #usage_grp
cond_prob_usage=np.round(pd.
    ↪crosstab(df['Usage_group'],df['Product'],normalize=True,margins=True),2)
cond_prob_usage
```

```
[182]: Product      KP281  KP481  KP781   All
Usage_group
Infrequent     0.11    0.08    0.00  0.18
Regular        0.21    0.17    0.01  0.38
Frequent       0.12    0.07    0.10  0.29
```

Power	0.01	0.02	0.12	0.14
All	0.44	0.33	0.22	1.00

Marginal Probability of Users Usage buying Threadmill:

- The Probability of threadmill being purchased by Infrequent User(0-2) is 18%
- The Probability of threadmill being purchased by Regular User(2-3) is 38%
- The Probability of threadmill being purchased by Frequent User(3-4) is 29%
- The Probability of threadmill being purchased by Power User(above 4) is 14%

Conditional Probability:

- The conditional probability of purchasing the treadmill model given that the customer is Infrequent User is - For Treadmill model KP281 11% For Treadmill model KP481 - 8% For Treadmill model KP781 - 0%
- The conditional probability of purchasing the treadmill model given that the customer is Regular User is - For Treadmill model KP281 21% For Treadmill model KP481 - 17% In []: Out []: For Treadmill model KP781 - 1%
- The conditional probability of purchasing the treadmill model given that the customer is Frequent User is - For Treadmill model KP281 12% For Treadmill model KP481 - 7% For Treadmill model KP781 - 10%
- The conditional probability of purchasing the treadmill model given that the customer is Power User is - For Treadmill model KP281 - 1% For Treadmill model KP481 - 2% For Treadmill model KP781 - 12%

```
[183]: #fitness_grp
cond_prob_fitness=np.round(pd.
↳crosstab(df['Fitness_group'],df['Product'],normalize=True,margins=True),2)
cond_prob_fitness
```

```
[183]: Product      KP281  KP481  KP781   All
Fitness_group
Poor           0.08   0.07   0.00  0.16
Below Average  0.30   0.22   0.02  0.54
Average        0.05   0.04   0.04  0.13
Excellent      0.01   0.00   0.16  0.17
All            0.44   0.33   0.22  1.00
```

Marginal Probability

- Marginal probabilities tell us the overall popularity of each product across all fitness groups.
- Treadmills (KP281) are the most popular, selected in 44% of all purchases.
- Dumbbells (KP481) are next with 33%, showing strong overall interest.

- Yoga Mats (KP781) account for 22%, making them the least selected overall—but with a niche user base (see conditional insights).

0.24 Insight:

- Treadmills dominate overall product choice, likely appealing to users beginning or maintaining cardio routines.

Conditional Probability Poor Fitness Group (16% of all users)

- Treadmills: 8%, Dumbbells: 7%, Yoga Mats: 0%

Insight:

- Users in poor fitness conditions show slight interest in Treadmills and Dumbbells, but no engagement with Yoga Mats. This group may be early in their fitness journey and focused on basic strength or cardio.

Below Average Fitness Group (54% of all users – the largest group)

- Treadmills: 30%, Dumbbells: 22%, Yoga Mats: 2%

Insight:

- This is the most active segment, with a clear preference for Treadmills, followed by Dumbbells. Yoga Mats are nearly unused here, suggesting flexibility-focused workouts aren't a priority yet.

Average Fitness Group (13% of all users)

- Treadmills: 5%, Dumbbells: 4%, Yoga Mats: 4%

Insight:

- Product choice is evenly distributed, suggesting these users are diversifying their fitness routines across cardio, strength, and flexibility.

Excellent Fitness Group (17% of all users)

- Treadmills: 1%, Dumbbells: 0%, Yoga Mats: 16%

Insight:

- Strong preference for Yoga Mats in this group, with zero interest in Dumbbells, and minimal use of Treadmills. Indicates that users in peak fitness levels are likely engaging in yoga, recovery, or core-focused training routines.

0.25 Recommendations

Tailored Product Positioning

- KP281: Market as a beginner's treadmill for students and early-career professionals.
- Highlight ease of use and affordability.

- KP481: Position as a balanced option for regular fitness users.
- Promote through office wellness programs and couples' packages.
- KP781: Brand as a premium product for serious runners and fitness enthusiasts.
- Include high-end features and elite fitness tracking.

Segment-Based Marketing Campaigns

- Use age, income, and fitness levels to personalize promotions.
- Example: Target users aged 25–35 with KP481 via social ads; users above 35 with KP781 via email campaigns or fitness apps.

Product Bundles & Add-Ons

- Offer accessory bundles with KP281 (starter kits).
- With KP781, provide premium services like home installation, personal coaching apps, or performance diagnostics tools.

Loyalty and Engagement Programs

- Reward frequent users and high-mileage customers with discounts or upgrades.
- Create gamified programs where customers can earn points based on miles run or usage consistency.

Referral & Group Purchase Incentives

- Encourage partnered buyers with couple or family treadmill bundles.
- Launch referral bonuses targeting loyal KP281 and KP481 users.

Data-Driven Personalization

- Implement an interactive product recommendation tool on the AeroFit website.
- Collect quick inputs (age, fitness level, workout frequency) to recommend the best product.

0.26 Insights

Product Preferences

- KP281 is the most purchased treadmill (44.4%) – favored for its affordability and appeal to beginners.
- KP481 (33.3%) and KP781 (22.2%) have niche yet stable market segments, with KP781 users having the highest fitness and income levels.

Demographics

- Age: Majority of customers are between 18 to 33 years, with a concentration around age 25–30.
- Gender: 57.8% of buyers are male, but females (42.2%) also represent a significant portion.
- Marital Status: 59.4% are partnered, suggesting that household purchase decisions may influence treadmill selection.

Education & Income

- Average education level is around 15.6 years, mostly secondary to higher education.
- Income ranges from \$30,000 to \$100,000+, with average income at \$53,700. KP781 buyers have higher income than others.

Usage Patterns

- Most customers use treadmills 3–4 times a week.
- Fitness level is self-rated as 3 out of 5 on average.
- Customers who work out more frequently also cover more weekly miles.

Product Choice by Segments

- KP281 users: Young adults with low income and below-average fitness.
- KP481 users: Adults with average fitness and moderate income.
- KP781 users: Middle-aged, high-income individuals with excellent fitness.